

CLAUDE CODE + CODEX

Modern Development Stack 2026

AI destekli geliştirme ortamını terminalden test ve gözlemlenebilirliğe kadar baştan aşağı modernize etme rehberi

Merkez yaklaşım: Claude Code lead/orchestrator ve implementer; Codex bağımsız reviewer ve problem solver. Graphify, Serena, Context7, Playwright, mise ve test katmanı iki ajan tarafından paylaşılan repo altyapısıdır.

Hedef ortam

- Enterprise React frontend
- .NET 8 backend / servisler
- Büyük ve uzun ömürlü kod tabanları
- Paralel agent geliştirme
- Düşük sürtünmeli terminal iş akışı

Araştırma özeti • Eylül 2026

1. Yönetici Özeti

Graphify doğru bir başlangıçtır; fakat tek başına kod tabanı bağlamını tamamlamaz. Claude Code ve Codex birlikte kullanıldığında en yüksek verim, iki ajanı aynı işi yapan yedekler gibi değil; farklı roller üstlenen bağımsız mühendisler gibi konumlandırınca elde edilir. Claude Code lead/orchestrator ve implementer; Codex ise bağımsız reviewer, ikinci görüş ve paralel problem solver olarak kullanılabilir. Graphify, Serena, Context7, Playwright, mise ve test katmanı her iki ajanın paylaştığı repo altyapısı olmalıdır.

Temel prensip: AI değişebilir; repository development contract değişmemeli. Her iki ajan da aynı AGENTS.md kurallarını, mise task'larını, testleri, security kontrollerini ve worktree izolasyonunu kullanmalı. MCP'yi yalnız dış sistem veya zengin bağlam gerçekten gerektiğinde aç.

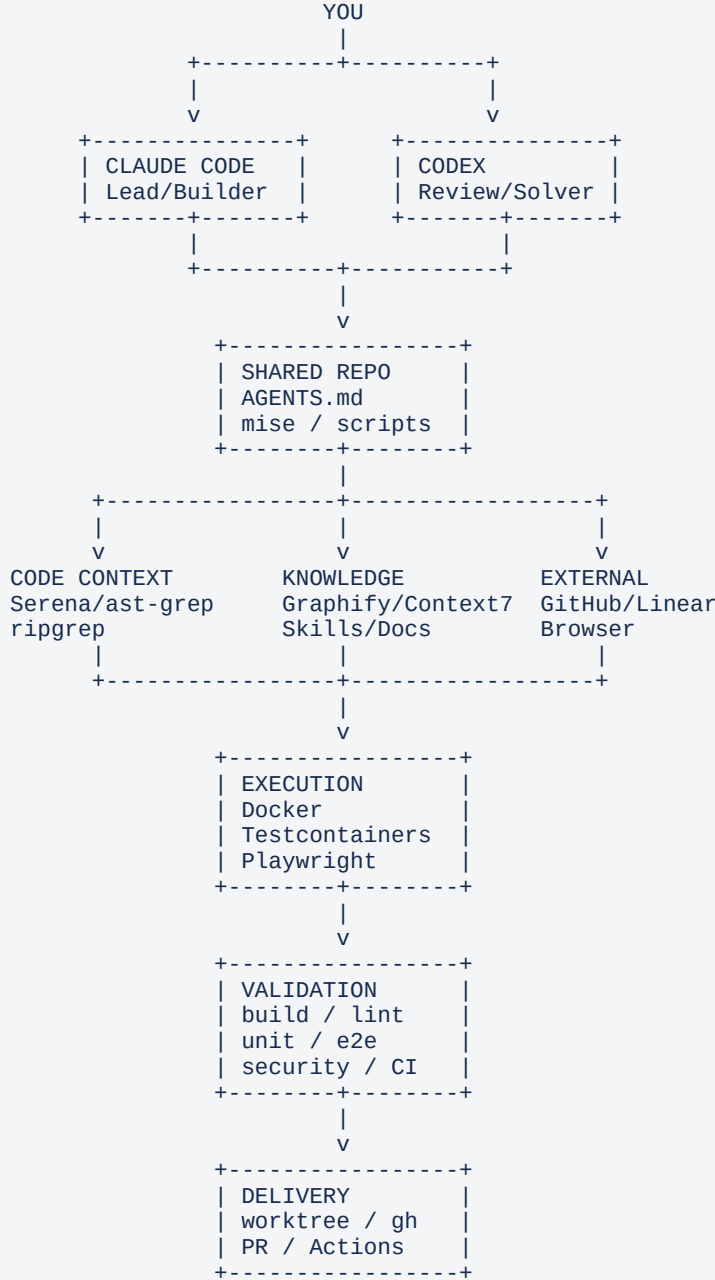
Önerilen çekirdek stack

Araç	Ana rol
Claude Code native	Orkestrasyon: Skills, Hooks, Subagents, Worktrees
Graphify	Mimari ilişkiler ve persistent knowledge graph
Serena	Symbol/LSP seviyesinde kod bulma, referans ve refactor
Context7	Güncel framework ve kütüphane dokümantasyonu
Playwright CLI + Skill	Browser test, debug ve UI doğrulaması
mise	Runtime sürümleri, env ve task standardizasyonu
Atuin	Terminal geçmişi ve agent/user komut hafızası
gh + modern CLI seti	PR/CI ve terminal navigasyonu
Testcontainers	Gerçek PostgreSQL/Redis ile integration test
OpenTelemetry	Production trace/metric/log standardı

En önemli karar

Büyük repo üzerinde tek çalışma dizininde birden fazla AI agent session çalıştırma. En güvenli model: 1 task = 1 branch = 1 worktree = 1 agent session. Claude Code ve Codex aynı feature üzerinde çalışacaksa bile doğrudan aynı working tree'de eşzamanlı edit yapmamalı.

2. AI-Ready Repo Mimarisi: Claude Code + Codex



Graphify + Serena neden birlikte?

Graphify sistemin mimari haritasını ve ilişkilerini çıkarır. Serena ise “bu symbol nerede tanımlı, kim referans veriyor, hangi implementation’lar var?” gibi IDE/LSP seviyesinde görevleri çözer. Birbirlerinin alternatifi değil, tamamlayıcıdır.

```

Graphify -> architecture / relationships / knowledge graph
Serena   -> symbols / references / refactor
ripgrep  -> fast raw text search
ast-grep -> structural AST search + codemod
  
```

3. Araç Öncelik Matrisi

Öncelik	Araç	Rol	Karar
S	Claude Code Skills / Hooks / Subagents	AI workflow motoru	Kesinlikle
S	Graphify	Repo knowledge graph	Kullanımda - doğru seçim
S	Serena	Symbol/LSP kod zekası	Kesinlikle
S	Context7	Güncel framework dokümantasyonu	Kesinlikle
S	Playwright CLI + Skill	Browser test/debug/validation	Kesinlikle
S	mise	Tool/version/env/task standardı	Kesinlikle
S	Atuin	Akıllı terminal ve agent geçmişi	Kesinlikle
S	GitHub CLI (gh)	PR/CI/issues/releases	Kesinlikle
S	Claude --worktree	Paralel agent izolasyonu	Kesinlikle
A	ast-grep	AST arama / codemod	Çok güçlü
A	ripgrep + fd + fzf	Search/navigation	Kur
A	delta	Okunabilir git diff	Kur
A	lazygit	Git TUI	Kur
A	Testcontainers	Gerçek bağımlılıkla integration test	.NET için çok değerli
A	Renovate	Dependency otomasyonu	Enterprise repo için evet
A	OpenTelemetry	Trace/metric/log	Production için evet
A	Linear MCP	AI - proje yönetimi	Linear varsa
A	GitHub MCP	AI - GitHub bağlamı	Seçici kullan
B	Worktrunk	Çoklu worktree UX	5+ paralel agent varsa
B	Graphite CLI	Stacked PR	PR hacmi artarsa
B	Dev Containers	Reproducible dev env	Ekip büyüdükçe
B	Ghostty	Modern terminal	Opsiyonel

4. Claude Code'u Önce Platform Haline Getir

Claude Code'u yalnızca prompt verilen bir terminal chatbotu gibi kullanmak, mevcut kapasitesinin büyük bölümünü boşa bırakır. Repo içinde açık bir AI çalışma sözleşmesi kur.

Önerilen .claude yapısı

```
.claude/
├── settings.json
├── rules/
│   ├── architecture.md
│   ├── backend.md
│   ├── frontend.md
│   └── testing.md
├── skills/
│   ├── architecture-review/
│   ├── bug-investigation/
│   ├── feature-development/
│   ├── database-migration/
│   └── release/
└── agents/
    ├── backend-architect.md
    ├── frontend-architect.md
    ├── security-reviewer.md
    ├── test-engineer.md
    └── code-reviewer.md
```

Rules, Skills, Hooks ayrımı

- CLAUDE.md / rules: kalıcı prensipler, mimari kısıtlar, repo sözleşmesi.
- Skills: Claude'un karar vermesi gereken tekrar kullanılabilir uzmanlık ve workflow'lar.
- Hooks: her seferinde uygulanması gereken deterministik enforcement. Örn. lint, typecheck, test, secret scan.

Pratik kural: "Claude bunu bazen hatırlasın" = rule/skill. "Claude bunu atlayamasın" = hook.

Subagents

Test logları, uzun araştırma, güvenlik review veya frontend/backend ayrışması gibi yüksek hacimli işleri ayrı context'e taşı. Ana Claude yalnızca özet ve kararları alsın.

Worktrees

```
claude --worktree feature-auth
claude --worktree bug-payment
```

Her agent'a ayrı filesystem/branch alanı vererek birbirlerinin değişikliklerini ezmesini engelle.

5. Kod Bağlamı ve Güncel Bilgi Katmanı

Serena

Serena, coding agent için IDE benzeri sembolik navigasyon sağlar. Büyük repo üzerinde dosya metni okumak yerine symbol, reference ve implementation düzeyinde çalışmak context tüketimini ve yanlış değişiklik riskini azaltır.

- Tanım ve referans bulma
- Interface implementation analizi
- Symbol seviyesinde güvenli refactor
- Büyük repo üzerinde hedefli retrieval

Context7

React, .NET, EF Core, Vite, TanStack, OpenTelemetry ve benzeri hızla değişen teknolojilerde modelin eski bilgiyle API üretmesini azaltmak için güncel dokümantasyonu çalışma anında getirir.

Kullanım mantığı: Framework/API sorusu → önce güncel docs → sonra implementation. Böylece deprecated pattern üretme riski düşer.

Playwright CLI + Skill

Frontend agent'ın yalnızca kod yazması değil, yaptığı değişikliği gerçek tarayıcıda doğrulaması gerekir. Playwright ile login, form doldurma, navigation, console/network kontrolü ve E2E doğrulama otomatikleşir.

```
code change
-> browser open
-> login / navigate
-> interact
-> console + network check
-> expected UI assertion
-> fix if needed
```

Agent görevlerinde varsayılan olarak CLI/Skill yaklaşımı token açısından daha yalın olabilir; uzun süreli stateful browser keşfi gerekiyorsa MCP mantıklıdır.

6. Terminal Verimlilik Stack'i

mise - geliştirme ortamının tek girişi

nvm/pyenv/asdf/env/task runner gibi parçalı araçların önemli bölümünü tek repo sözleşmesinde birleştirir. AI açısından daha da değerlidir: agent hangi test komutunu arayacağını tahmin etmez; tek standart task interface kullanır.

```
[tools]
node = "24"
dotnet = "8"

[tasks.test]
run = "dotnet test && pnpm test"

[tasks.check]
run = "dotnet build && pnpm lint && pnpm typecheck"
```

AI-dostu repo API'si: mise run setup | dev | test | lint | check | e2e

Atuin - terminal hafızası

Komutları yalnızca string olarak değil; directory, session, exit status, duration ve agent/user gibi bağlamlarla tutar. Böylece geçmişte bu repo için hangi komutların gerçekten çalıştığını bulmak kolaylaşır.

Modern CLI temel seti

ripgrep (rg): çok hızlı source search

fd: find yerine daha sade dosya arama

fzf: fuzzy selector

zoxide: alışkanlığa göre akıllı cd

bat: syntax-highlighted cat

delta: okunabilir ve side-by-side git diff

lazygit: terminal Git TUI

gh: PR, checks, Actions, issues ve releases

7. Kalite, Test ve Production Katmanı

ast-grep

Regex yerine AST yapısına göre arama ve rewrite yapar. Repo-wide migration ve codemod işlerinde özellikle agent'ların güvenli toplu değişiklik yapması için değerlidir.

Testcontainers for .NET

Mock veritabanı yerine test sırasında gerçek PostgreSQL, Redis ve benzeri bağımlılıkları geçici container olarak ayağa kaldırır. "Mockta geçti, production'da bozuldu" sınıfındaki hataları azaltır.

```
Test starts
-> PostgreSQL container
-> Redis container
-> integration tests
-> containers destroyed
```

React test piramidi

```
Vitest      -> unit / hook / utility
Storybook   -> component states / interactions
Playwright  -> gerçek kullanıcı journey / E2E
```

Renovate

NuGet, npm, Docker ve GitHub Actions dahil bağımlılıkları gruplama ve zamanlama kurallarıyla otomatik yönet. Security update'leri hızlı, major update'leri kontrollü, ecosystem güncellemelerini gruplu tut.

Secret scanning

AI agent'ların config ve shell erişimi olduğu ortamda GitHub Push Protection ve lokal/CI Gitleaks gibi kontroller önemli bir emniyet kemeridir.

OpenTelemetry

Production debugging'i log satırı aramak yerine trace + metric + log korelasyonuna taşı. .NET servislerde request → DB → Redis → external service zincirini tek trace üzerinden izlemek, Claude ile kök neden analizini de ciddi biçimde iyileştirir.

8. GitHub, Linear ve Paralel Agent Ölçeklemesi

GitHub CLI önce, GitHub MCP gerektiğinde

Claude'un rahatça yapabildiği PR/check/issue işlemlerinde gh küçük ve deterministik bir arayüz sağlar. Zengin repository bağlamı veya MCP tool akışı gerçekten gerekiyorsa GitHub MCP'yi yalnızca gerekli toolset'lerle aç.

Linear MCP

Linear kullanılıyorsa issue → implementation → test → PR → issue update zincirini Claude Code'a bağlamak için doğal bir katmandır.

```
Linear Issue
-> Claude Code
-> Graphify + Serena
-> implementation
-> tests
-> GitHub PR
-> Linear update
```

Worktrunk

Claude native worktree 1-3 paralel agent için yeterlidir. Sürekli 5-10+ worktree yönetmeye başladığında Worktrunk gibi bir UX katmanı anlamlı hale gelir.

Graphite CLI

AI tarafından üretilen dev changeset'leri tek 4000 satırlık PR yerine birbirine bağlı, review edilebilir stacked PR'lara bölmek için değerlidir. İlk gün gerekmez; PR hacmi yükseldiğinde ekle.

Dev Containers

Ekip büyüdükçe "bende çalışıyor" farklarını azaltmak için OS, araç, bağımlılık ve editor ortamını repo tarafından tanımlanan reproducible container'a taşı.

9. Önerilen Kurulum Sırası

Dalga 1 - hemen

- Graphify (mevcut)
- Serena
- Context7
- Playwright CLI + Skill
- mise
- Atuin
- zoxide + fzf + fd + ripgrep + bat + delta + lazygit + gh

Dalga 2 - Claude Code'u platformlaştır

- CLAUDE.md sadeleştirme
- .claude/rules/
- .claude/skills/
- .claude/agents/
- Hooks: lint + typecheck + tests + secret scan
- Claude --worktree

Dalga 3 - kalite ve güvenilirlik

- Vitest
- Storybook
- Playwright E2E
- Testcontainers
- Renovate
- Secret scanning
- OpenTelemetry

Dalga 4 - ölçek ve entegrasyon

- Linear MCP
- GitHub MCP
- Worktrunk
- Graphite stacked PR
- Claude agent teams
- Dev Containers

Ne zaman Agent Teams?

Büyük cross-layer feature, mimari araştırma veya birbirinden bağımsız hipotezlerin paralel test edilmesi gibi görevlerde kullan. Küçük bug veya tek dosyalık değişiklikte token ve koordinasyon overhead'i gereksizdir.

10. Sadece 10 Şey Seçecek Olsaydım

#	Araç	Neden
1	Claude Code Skills + Hooks + Subagents	Agent davranışını repo standardına dönüştürür.
2	Graphify	Sistemin mimari haritasını kalıcı hale getirir.
3	Serena	Symbol/reference seviyesinde kod zekası sağlar.
4	Context7	Güncel framework dokümantasyonunu çalışma anına getirir.
5	Playwright CLI + Skill	Agent'ın gerçek UI'yı test etmesini sağlar.
6	mise	Tek ve deterministik dev/test/task interface oluşturur.
7	Atuin	Terminal ve agent komut geçmişini gerçek hafızaya dönüştürür.
8	Claude --worktree	Paralel geliştirmeyi filesystem seviyesinde izole eder.
9	gh + rg/fd/fzf	GitHub ve terminal sürtünmesini ciddi azaltır.
10	Testcontainers + güçlü test katmanı	AI-generated code'u gerçek bağımlılıklar üzerinde doğrular.

Özet kombinasyon: Graphify sistemi anlar. Serena kodun sembolik yapısını anlar. Context7 bugün kullanılan API'yi bilir. Playwright yapılan değişikliği gerçek uygulamada doğrular. mise + Hooks + Testcontainers + Worktrees bu zekâyı güvenilir bir mühendislik platformuna dönüştürür.

Codex notu: Bu Top 10 listesi ek altyapı seçimlerini sıralar; zaten kullandığın Codex ayrı bir çekirdek AI ajanıdır. Önerilen modelde Claude Code ve Codex aynı shared repository infrastructure üzerinde farklı roller üstlenir.

Önerilen günlük çalışma akışı

1. Linear/GitHub issue seç
2. `claude --worktree <task>`
3. Graphify ile mimari bağlam
4. Serena ile symbol/reference analizi
5. Context7 ile güncel API doğrulama
6. Claude implementation
7. Hooks + mise run check
8. Testcontainers / Vitest / Playwright
9. `gh pr create + checks`
10. Review -> merge -> worktree cleanup

11. Claude Code + Codex Birlikte Kullanım Modeli

Claude Code ile Codex'i yan yana kullanmanın asıl avantajı, ikisine sürekli aynı feature'ı yazdırmak değildir. Daha güçlü model; birinin planlayıp uyguladığı işi diğerinin bağımsız varsayımlarla incelemesi, zor problemlerde farklı hipotezler üretmeleri ve her ikisinin de aynı repository contract üzerinde çalışmasıdır.

Varsayılan rol dağılımı: Claude Code = lead/orchestrator + implementer. Codex = independent reviewer + second opinion + paralel problem solver. Bu roller sabit dogma değildir; gerçek repo performansına göre zamanla değiştirilebilir.

Rol matrisi

İş	Claude Code	Codex
Büyük feature geliştirme	Primary	Primary / paralel alternatif
Mimari analiz	Primary	Second opinion
Repo exploration	Primary	Second opinion
Refactor	Primary	Independent verification
Zor bug analizi	Hipotez A	Hipotez B
Unit / integration test	Evet	Evet
Code review	Evet	Primary independent reviewer
Security / tenant review	İlk kontrol	Bağımsız kontrol
PR inceleme	Evet	Özellikle güçlü kullanım
Alternatif implementation	Evet	Evet
Paralel işler	Evet	Evet

En güçlü pattern: biri yazar, diğeri inceler

```

CLAUDE CODE
-> planla
-> implement et
-> testleri çalıştır
    |
    v
CODEX
-> diff'i bağımsız incele
-> architecture / security / performance / tests
-> severity ile bulguları raporla
    |
    v
CLAUDE CODE
-> bulguları doğrula
-> yalnız geçerli olanları düzelt
    |
    v
CI / PR / MERGE

```

Aynı modele "kendi kodunu tekrar review et" demek faydalıdır ama bağımsız bir ikinci modelin farklı varsayımlarla değerlendirmesi daha güçlü bir hata yakalama katmanı oluşturabilir. Özellikle authorization, tenant isolation, concurrency, N+1, API compatibility ve React state problemlerinde bu çapraz review standardize edilebilir.

Zor buglarda iki bağımsız hipotez üret

Production bug veya belirsiz root-cause durumunda iki ajanı da önce yalnız analiz modunda çalıştır. Kod değiştirmeden birbirinden bağımsız kök neden hipotezi oluştursunlar. Ardından hipotezleri birbirlerine çürüttür veya test suite üzerinden kanıtlar.

```
Bug: bazen duplicate Order oluşuyor

Claude Code -> "transaction / retry boundary olabilir"
Codex        -> "idempotency eksik olabilir"

Sonraki adım:
-> her hipotez için kanıt toplama
-> regression test yaz
-> yalnız kanıtlanan root cause'u düzelt
```

Graphify, Serena ve diğer araçlar iki ajan arasında ortak olmalı

Graphify, Serena, Context7, Playwright, mise, ast-grep ve test altyapısını Claude'a özel eklentiler gibi değil, repository development infrastructure olarak düşün. Amaç; hangi ajan çalışırsa çalışsın aynı kod bağlamına, aynı güncel dokümana ve aynı doğrulama komutlarına ulaşabilmesidir.

```
+-- Claude Code
|
AI-ready repo ---+-- shared tools / tests / docs
|
+-- Codex
```

AGENTS.md + CLAUDE.md ayrımı

Kritik mühendislik gerçeklerini yalnız CLAUDE.md içine koyma. İki ajanlı ortamda tool-independent repository contract için kökte AGENTS.md kullanmak daha sağlıklı bir modeldir. Claude'a özgü davranış, delegation ve tool tercihleri ise CLAUDE.md ve .claude/ altında kalabilir.

```
repo/
|-- AGENTS.md           # iki ajanın ortak repo sözleşmesi
|-- CLAUDE.md           # Claude Code'a özgü davranış
|-- docs/
|   |-- architecture/
|   +-- decisions/
|-- .claude/
|   |-- rules/
|   |-- skills/
|   +-- agents/
|-- .agents/
|   +-- skills/
|-- scripts/
|   |-- setup
|   |-- check
|   |-- test
|   |-- test-e2e
|   +-- validate
+-- mise.toml
```

- AGENTS.md: architecture, coding conventions, Definition of Done, testing policy, database/tenant/security kuralları ve ortak komutlar.
- CLAUDE.md: Serena/Graphify kullanımı, subagent delegation, Claude'a özel tool ve workflow tercihleri.

Tek komut sözleşmesi: AI değişir, task interface değişmez

Claude Code ve Codex farklı komutlar tahmin etmemeli. Repo her ikisine de aynı küçük, deterministik interface'i sunmalı.

```
mise run setup
mise run dev
mise run check
mise run test
mise run e2e
mise run validate
```

Repository contract: Agent değiştirdiğinde development workflow değişmiyorsa doğru tasarlamışsın demektir.

Worktree modeli: aynı working directory'de yarışma

Claude Code ve Codex'i mümkün olduğunca aynı working directory üzerinde eşzamanlı değişiklik yapacak şekilde çalıştırma. Her task için ayrı branch + worktree + agent session kullan. Böylece dosya çakışması, yanlış diff ve context kirliliği azalır.

```
main
|
+-- worktree/customer-activity -> Claude Code
+-- worktree/payment-fix       -> Codex
+-- worktree/report-performance -> Claude Code
+-- worktree/test-hardening     -> Codex
```

Pratik kural: 1 task = 1 branch = 1 worktree = 1 agent session. Aynı feature üzerinde çapraz review gerekiyorsa reviewer önce diff/branch'i incelemeli; doğrudan aynı working tree'de eşzamanlı edit yapmamalı.

Önerilen günlük dual-agent akışı

1. Issue seç
2. Claude Code -> repo bağlamını çıkar + implementation plan oluştur
3. Codex -> planı eleştir; riskleri ve eksikleri bul
4. Claude Code -> geçerli bulgularla planı güncelle + implement et
5. mise / hooks -> build + lint + unit + integration + e2e
6. Codex -> tamamlanmış diff'i bağımsız review et
7. Claude Code -> Codex bulgularını doğrula ve geçerli olanları düzelt
8. mise run validate / CI
9. gh pr create
10. Final human review -> merge

Bu modelde Claude Code planlama ve uygulama döngüsünü taşıırken Codex bağımsız kalite katmanı olur. İki ajan arasındaki anlaşmazlıklar "hangi model daha iyi?" tartışmasıyla değil, test, runtime kanıtı, architecture decision ve requirement üzerinden çözülür.

Ne zaman iki ajanı da aynı probleme koşmalısın?

- Root cause'u belirsiz ve production etkisi yüksek buglar.
- Authorization, tenant isolation veya güvenlik açısından kritik değişiklikler.
- Büyük veri modeli / migration / concurrency değişiklikleri.
- Birden fazla makul mimari çözüm bulunan feature'lar.
- Çok büyük refactor veya framework migration'ları.

Basit CRUD, typo, tek dosyalık küçük değişiklik veya zaten güçlü testlerle çevrili rutin işler için iki ajan kullanmak gereksiz token ve koordinasyon maliyeti yaratabilir.

Not: Araç eklemek tek başına 10x verim sağlamaz. Asıl kazanç; repo için küçük ve deterministik task arayüzü, güçlü test enforcement, doğru context retrieval ve paralel agent izolasyonunun birlikte uygulanmasından gelir.

12. Kurulum Öncesi Araç Doğrulama ve Araştırma Gate'i

Bu rehberde adı geçen Graphify, Serena, Context7, mise, Atuin, Playwright ve diğer araçlar bir "kurulacaklar listesi" değildir. Her biri yalnızca candidate tool'dur. Claude Code veya Codex, bir aracı popüler veya modern olduğu için değil, repository'de kanıtlanmış bir ihtiyacı en düşük toplam maliyetle çözdüğü için seçmelidir.

Temel kural: Önce problemi kanıtla → mevcut çözümü incele → resmî kaynaktan adayı doğrula → ciddi alternatifleri karşılaştıır → fayda/maliyet yeterliyse kur. "Hiçbir şey kurmamak" geçerli bir karardır.

Need Validation Gate - önce ihtiyacı kanıtla

Bir araç seçmeden önce repository'den somut evidence çıkar. "Best practice", "AI-friendly", "enterprise-grade" veya "popüler" tek başına gerekçe değildir.

Need:
Evidence from repository:
Current solution:
Observed gap:
Expected benefit:
Operational / context / security cost:
Decision: INSTALL | KEEP EXISTING | REJECT | LATER

Örnek: CI Node 24 kullanıyor, README Node 22 diyor ve hiçbir version pinning yoksa tool/version manager ihtiyacı gerçektir. Buna karşılık .tool-versions veya başka bir mevcut çözüm CI ile tutarlı çalışıyorsa sırf listede mise bulunduğu için migration yapılmamalıdır.

Official-Source-First Gate - bilinen aracı yeniden keşfetme

Aşağıdaki Verified Source Registry'de bir araç için canonical adres verilmişse Claude veya Codex o aracın kimliğini, kurulumunu ve güncel davranışını bulmak için genel web aramasıyla başlamamalıdır. Önce tam olarak verilen resmî kaynaklara gitmelidir.

- Kurulum talimatı: resmî docs veya resmî repository README.
- Güncellik: releases / changelog / tags.
- Güvenlik: security policy / advisories ve son düzeltmeler.
- Uyumluluk: mevcut OS, dil, runtime, Claude Code, Codex, CI ve worktree akışı.
- Genel web araması: yalnızca ciddi alternatifleri keşfetmek veya üçüncü taraf saha deneyimini karşılaştırmak için.

Alternative Research Gate - verilen araca bias gösterme

İhtiyaç doğrulandıktan sonra problem alanında makul alternatifler varsa en az 3 ciddi adayı karşılaştıır. Örneğin "symbol-aware code navigation" ihtiyacı varsa soru "Serena nasıl kurulur?" değil, "Bu repository için 2026 itibarıyla en uygun symbol-aware agent tooling hangisi?" olmalıdır. Serena yalnızca adaylardan biridir.

Problem / capability
Candidate A | Candidate B | Candidate C
repo fit | language support | accuracy
Claude/Codex support | local/offline | token/context overhead
maintenance activity | security | install complexity | CI fit
Winner / Keep existing / Do nothing / Defer

Security Gate - özellikle MCP ve agent tooling

Agent araçları shell, filesystem, browser veya remote MCP erişimi alabildiği için yalnız feature listesine bakmak yeterli değildir. Kurulumdan hemen önce security/advisory sayfaları kontrol edilmeli ve yalnız patched/current sürüm tercih edilmelidir. Örneğin Serena'nın GitHub Security alanında 2026'da yayımlanmış advisories bulunduğu için sürüm ve mitigation doğrulaması atlanmamalıdır.

Source hierarchy: Official docs > official repository > releases/changelog > security/advisories > issue tracker. Üçüncü taraf blog veya tutorial kurulum için source of truth değildir.

13. Verified Source Registry - Doğrudan Resmî Kaynak Kataloğu

Bu katalog, araç adını ve resmî kaynağını yeniden aramakla zaman kaybetmemek için hazırlanmıştır. Bilinen bir candidate değerlendirildiğinde önce buradaki adresler açılır. Alternatif discovery gerekiyorsa bundan sonra web araştırması yapılır. URL'ler Eylül 2026 itibarıyla resmî proje/ürün kaynakları olarak doğrulanmıştır.

A. Ana AI ajanları ve repository contract

Araç	Rol / neden	Canonical resmî kaynaklar
Claude Code	Lead / orchestrator / implementer	Docs: https://code.claude.com/docs/en/overview Repo: https://github.com/anthropics/claude-code Security: https://github.com/anthropics/claude-code/security
OpenAI Codex	Independent engineer / reviewer / parallel solver	CLI docs: https://developers.openai.com/codex/cli AGENTS.md: https://developers.openai.com/codex/guides/agents-md Repo: https://github.com/openai/codex Product: https://openai.com/codex/
GitHub CLI (gh)	PR / Actions / issues / releases	Docs: https://docs.github.com/en/github-cli Manual: https://cli.github.com/manual/gh Repo: https://github.com/cli/cli
GitHub MCP Server	GitHub tool integration for agents	Repo: https://github.com/github/github-mcp-server
Linear MCP	Issue / project integration	Docs: https://linear.app/docs/mcp

B. Codebase context ve güncel dokümantasyon

Araç	Rol / neden	Canonical resmî kaynaklar
Graphify	Architecture / relationship knowledge graph	Repo: https://github.com/Graphify-Labs/graphify Releases: https://github.com/Graphify-Labs/graphify/releases Org: https://github.com/Graphify-Labs
Serena	Symbol-aware / LSP code intelligence	Docs: https://oraio.github.io/serena/ Repo: https://github.com/oraio/serena Changelog: https://github.com/oraio/serena/blob/main/CHANGELOG.md Security: https://github.com/oraio/serena/security
Context7	Version-specific current library docs	Repo: https://github.com/upstash/context7 Releases: https://github.com/upstash/context7/releases Security docs: https://github.com/upstash/context7/blob/master/docs/security/overview.mdx
ast-grep	AST search / lint / codemod	Docs: https://ast-grep.github.io/ Repo: https://github.com/ast-grep/ast-grep

C. Browser, test ve validation altyapısı

Araç	Rol / neden	Canonical resmî kaynaklar
Playwright	E2E / browser validation / trace	Docs: https://playwright.dev/ .NET docs: https://playwright.dev/dotnet/ Repo: https://github.com/microsoft/playwright MCP: https://github.com/microsoft/playwright-mcp
Testcontainers .NET	Real dependency integration tests	Docs: https://dotnet.testcontainers.org/ Repo: https://github.com/testcontainers/testcontainers-dotnet
Vitest	Frontend unit / hook tests	Docs: https://vitest.dev/ Repo: https://github.com/vitest-dev/vitest
Storybook	Component states / interaction tests	Testing docs: https://storybook.js.org/docs/writing-tests/ Repo: https://github.com/storybookjs/storybook
OpenTelemetry .NET	Traces / metrics / logs	Docs: https://opentelemetry.io/docs/languages/dotnet/ Repo: https://github.com/open-telemetry/opentelemetry-dotnet

D. Environment, dependency ve reproducibility

Araç	Rol / neden	Canonical resmî kaynaklar
mise	Tool versions / env / canonical tasks	Docs: https://mise.jdx.dev/ Repo: https://github.com/jdx/mise
Renovate	Dependency automation	Docs: https://docs.renovatebot.com/ Repo: https://github.com/renovatebot/renovate
Dev Containers	Reproducible development environment spec	Spec: https://containers.dev/ Repo: https://github.com/devcontainers/spec CLI: https://github.com/devcontainers/cli
Docker Compose Watch	Local container development loop	Docs: https://docs.docker.com/compose/how-to/FILE-WATCH/

E. Parallel development ve PR workflow

Araç	Rol / neden	Canonical resmî kaynaklar
Worktrunk	Many-worktree / parallel-agent UX	Repo: https://github.com/max-sixty/worktrunk Docs: https://worktrunk.dev/
Graphite CLI	Stacked PR workflow	Docs: https://graphite.com/docs/cli-overview Install: https://graphite.com/docs/install-the-cli

F. Terminal productivity - developer-level tools

Araç	Rol / neden	Canonical resmî kaynaklar
Atuin	Shell history + agent-aware history	Docs: https://docs.atuin.sh/ Repo: https://github.com/atuinsh/atuin
ripgrep	Fast text/code search	Repo: https://github.com/BurntSushi/ripgrep
fd	Fast filesystem search	Repo: https://github.com/sharkdp/fd
fzf	Fuzzy selection	Repo: https://github.com/junegunn/fzf
bat	Readable cat / syntax highlight	Repo: https://github.com/sharkdp/bat
delta	Readable git diff	Repo: https://github.com/dandavison/delta
lazygit	Git TUI	Repo: https://github.com/jesseduffield/lazygit
zoxide	Smarter directory navigation	Repo: https://github.com/ajeetsouza/zoxide
Ghostty	Modern terminal emulator	Docs: https://ghostty.org/docs Repo: https://github.com/ghostty-org/ghostty

G. Security controls

Araç	Rol / neden	Canonical resmî kaynaklar
Gitleaks	Local / CI secret detection	Repo: https://github.com/gitleaks/gitleaks
GitHub Secret Scanning	Hosted secret detection	Docs: https://docs.github.com/en/code-security/secret-scanning
GitHub Push Protection	Block secrets before push	Docs: https://docs.github.com/en/code-security/secret-scanning/introduction/about-push-protection

Katalog nasıl kullanılmalı?

KNOWN CANDIDATE

- > open exact official sources above
- > verify installation + releases + security
- > prove repo need
- > research alternatives only if the need exists
- > compare benefit / complexity / context / security
- > INSTALL | KEEP | REJECT | LATER

Sonuç: PDF içindeki isimler otomatik kurulum talimatı değil, doğrulanmış candidate registry'sidir. Claude Code veya Codex, aracın adını/repo'sunu yeniden keşfetmek için arama motorunda dolaşmak yerine önce canonical source'a gider; fakat "bu araç gerçekten gerekli mi?" ve "daha iyi alternatif var mı?" sorularını yine bağımsız olarak cevaplar.